

Database 2 emne oversikt

DAT2000 – USN Vestfold – Vår 2026 – Oppsummering av emnet

Innhold

Database 2 emne oversikt.....	1
Databaseadministrasjon (DBA) og databasesikkerhet.....	2
Databaseadministrasjon (DBA)	2
<i>Systemkatalogen</i>	2
Databasehåndteringssystem (DBHS)	3
Databasesikkerhet	4
Brukere, roller og brukeradministrasjon	7
Et grovmasket sikkerhetssystem	8
SQL	9
Database i sky	11
Lagrede programmer.....	12
Lagrede rutiner	12
Parametere	13
Bruk av pekere (Cursors)	13
Triggere	14
Forretningsregler	18
Transaksjoner og gjenoppretning	18
Transaksjonsloggen	19
ACID-egenskapene	20
Isolasjonsnivåer	20
Samtidighetskontroll	20
Filer, indekser og dataformer	22
Blokker	24
Indekser	25
Datatyper.....	28
PostgreSQL Storage Structure (Fra kapittel 9)	29

Hierarkiske dataformater.....	32
NoSQL	35

Databaseadministrasjon (DBA) og databasesikkerhet

Databaseadministrasjon (DBA)

Databaseadministrasjon deles ofte i:

- **Dataadministrasjon (strategisk):** Retningslinjer, prosedyrer, valg av DBMS.
- **Databaseadministrasjon (teknisk):** Daglig drift, sikkerhet, brukere, ytelse og backup.

DBAs hovedoppgaver

- **Installere og konfigurere DBMS**
- Bruker- og rolleadministrasjon
- Overvåking og optimalisering
- Sikkerhetskopiering (backup) og gjenoppretting (recovery)
- Kapasitetsplanlegging og sikkerhet

Systemkatalogen

Systemkatalogen er databasen sitt eget “oppslagsverk” som inneholder informasjon om strukturen, objektene og brukerne i systemet. Den fungerer som et internt register som DBMS bruker for å holde oversikt over alt som finnes og alt som er definert i databasen.

- Metadata, eller «data om data» er beskrivelser av databasen:
 - Hvilke tabeller finnes? Hvordan er tabellene bygget opp?
 - Hvilke indekser er definert?
- Statistiske data brukes av spørreoptimalisatoren:
 - Hvor mange rader har tabellene?
 - Hvor mange forskjellige verdier er lagret i en kolonne?
- Data om brukere og deres rettigheter.

Systemkatalogen er en samlebetegnelse slike data.

- Data i systemkatalogen er ofte lagret på tabellform (systemtabeller), som betyr at vi kan bruke SQL for rapportering.

Databasehåndteringssystem (DBHS)

Et DBHS (*Databasehåndteringssystem*) er programvaren som **lagrer, organiserer, beskytter og håndterer** data i en database. Det er «motoren» som gjør at databaser fungerer.

- Et DBHS (Databasehåndteringssystem) er *hele* programvaresystemet som gjør det mulig å lagre, organisere, sikre og hente data på en kontrollert og effektiv måte. Det er fundamentet som alle databaser bygger på — uten et DBHS ville en database bare vært en samling filer uten struktur, sikkerhet eller kontroll.

Noen av de vanligste DBHS programmene er **PostgreSQL, MySQL, Oracle Database, SQL Server** og **MongoDB** (NoSQL).

Databasehåndteringssystemers funksjoner

Et moderne DBHS, som PostgreSQL, består typisk av:

1. Lagringsmotor

- Lagrer data fysisk på disk eller i minne
- Håndterer sider, blokker, buffere og filstrukturer
- Sikrer at data lagres effektivt og sikkert

2. Spørringsmotor (Query Engine)

- Parser SQL
- Optimaliserer spørringer
- Velger beste indeks/plan
- Utfører spørringen

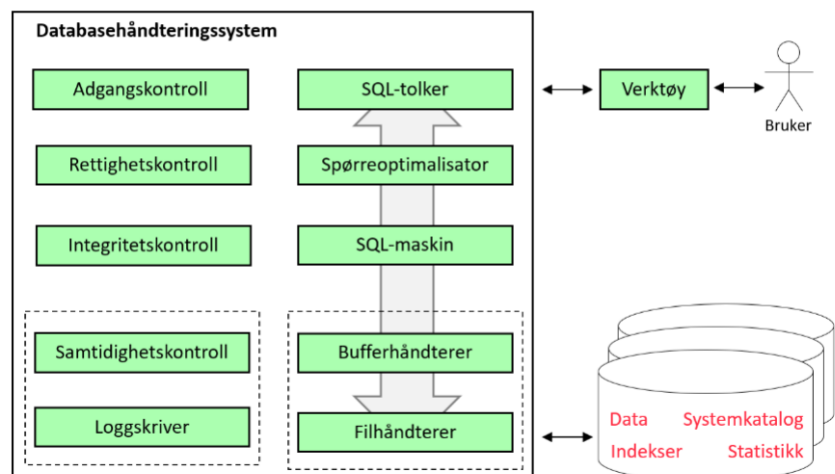
3. Transaksjonshåndtering

- ACID-egenskaper (krav som sikrer at transaksjoner i en database utføres korrekt, trygt og forutsigbart, selv ved feil, krasj eller samtidig bruk)
- Låsing, isolasjonsnivåer, samtidighetskontroll
- Write-Ahead Logging (WAL)

4. Bruker- og rettighetskontroll

- Roller, grupper, privilegier
- Autentisering og autorisasjon
- System- og objektrettigheter

5. Systemkatalog (metadata)



- Informasjon om tabeller, indekser, brukere, rettigheter
- DBHS bruker dette for optimalisering og sikkerhet

6. Backup og recovery

- Sikkerhetskopiering
- Gjenoppbygging etter feil
- Loggbasert recovery

7. Administrasjonsverktøy

- Opprette brukere
- Overvåke ytelse
- Starte/stoppe databasen
- Analysere SQL

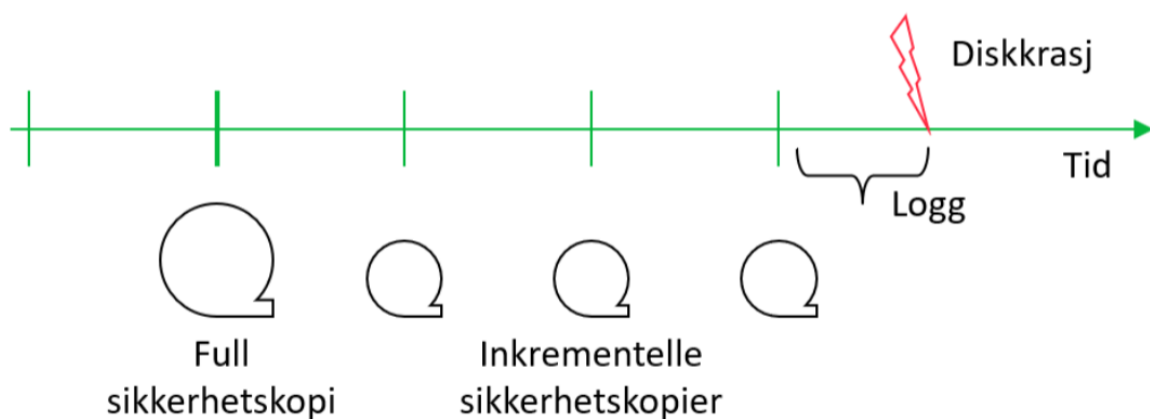
Databasesikkerhet

Databasesikkerhet er en samling av sikkerhetspolicyer og -prosedyrer, databegrensninger, sikkerhetsmetoder og sikkerhetsverktøy som er blandet sammen for å implementere alle nødvendige tiltak for å sikre konfidensialitet, integritet og tilgjengelighet til alle komponenter i databasemiljøet.

- Disse komponentene inkluderer personer, applikasjoner, nettverk, operativsystemer, databasehåndteringssystemer, datafiler og data.

Sikkerhetskopiering og gjenoppbygging

Sikkerhetskopiering og gjenoppbygging (recovery) handler om hvordan databasen kan **gjenopprettes til en konsistent tilstand** etter feil.



Typer sikkerhetskopiering:

- **Full backup** Hele databasen kopieres.
- **Inkrementell/differensiell backup** Kun endringer siden forrige full backup lagres.
- **Backup til skyen** Vanlig i moderne systemer.

Hvordan gjenopprette:

- Databasen bruker **siste full backup + transaksjonsloggen** for å gjenopprette databasen.
 - Loggen inneholder alle endringer som har skjedd etter siste backup. Logg brukes for å gjenopprette endringer som ikke ble lagret under siste sikkerhetskopiering.

Regler for gjenoppretting etter et krasj

1. Transaksjoner som er committed før checkpoint

- Disse er allerede skrevet til disk.
- Trenger ikke redo.

2. Transaksjoner som var aktive ved checkpoint eller startet etter

- Disse kan ha endringer som ikke er skrevet til disk.
- Redo alle som har både *start* og *commit* i loggen etter checkpoint.

3. Transaksjoner som var aktive ved krasjtidspunktet

- Disse er ikke fullført.
- Undo alle som ikke har commit-post i loggen.

Hva DBMS må tilby innen gjenoppretning

- Sikkerhetskopier tas jevnlig for å kunne gjenopprette databasen ved alvorlige feil.
- Loggføring registrerer alle transaksjoner og alle endringer som gjøres.
 - Loggen inneholder *start*, *commit/abort* og alle oppdateringer.
- Checkpoints brukes for å begrense hvor langt tilbake i loggen systemet må gå etter et krasj.

Hvorfor checkpoints er viktige:

- Etter en feil vet man ikke hvor langt bakover i loggen man må lete.
- Checkpoints markerer et punkt hvor alle tidligere transaksjoner er trygt skrevet til disk.

Dette reduserer:

- Mengden logg som må leses
- Antall transaksjoner som må gjøres redo eller undo

Checkpoints vs. Inkrementell backup:

- Checkpoint = loggmarkør for raskere crash recovery
- Inkrementell backup = fysisk kopi av data for å gjenopprette databasen

Sikkerhetstrusler og virkemidler

Vi skiller mellom **uhell**, **angrep** og hvilke **virkemidler** (tiltak) som brukes for å beskytte databaser og IT-systemer.

Uhell (utilsiktede hendelser)

Dette er ikke-angrep, men hendelser som kan føre til tap av data eller nedetid:

- Menneskelig feil
- Programvarefeil
- Maskinvarefeil
- Strømbrudd

Angrep (bevisste handlinger)

Trusler fra personer eller systemer som prøver å få innsyn eller endre data:

- Interne eller eksterne angripere
- Forsøk på innsyn eller manipulering
- Direkte angrep mot databasen
- Indirekte angrep via operativsystem eller nettverk

Virkemidler (sikkerhetstiltak)

Tiltak som reduserer risiko og konsekvenser av både uhell og angrep:

- Sikkerhetskopiering, loggføring og gjenoppbygging
- Overvåking
- Kryptering
- Aksesskontroll (fysisk sikkerhet, passord, rettigheter)

- Duplisering av utstyr (redundans)
- Nødaggregat
- Rutiner og «brannøvelser»

Brukere, roller og brukeradministrasjon

Brukeradministrasjon handler om hvordan et databasesystem styrer **hvem som får gjøre hva** i databasen. Det er en kjerneoppgave for en DBA og en viktig del av datasikkerhet.

- **Brukerinformasjon lagres i systemtabeller** – Databasen lagrer informasjon om brukere, roller og rettigheter i egne systemtabeller (metadata)-

Objekttrettigheter vs. systemrettigheter

- **Objekttrettigheter:** Rettigheter knyttet til konkrete databaseobjekter (f.eks. SELECT på tabellen *Kunder*).
- **Systemrettigheter:** Rettigheter som gjelder hele DBMS-et (f.eks. starte/stoppe databasen, opprette brukere).

Roller og grupper i databasesystemer

Bruk av roller og grupper er en sentral mekanisme for å organisere tilgangskontroll i et databasesystem. **I stedet for å gi rettigheter direkte til hver enkelt bruker, samles brukere i roller** som representerer funksjoner, avdelinger eller ansvarsområder. Dette gjør administrasjonen enklere, mer oversiktlig og mindre feilutsatt.

Roller - En rolle er en logisk enhet som kan **eie objekter** og **ha rettigheter**, på samme måte som en bruker. Rollen fungerer som en beholder for privilegier.

Kjennetegn:

- Kan tildeles rettigheter på tabeller, views, prosedyrer osv.
- Kan representere en jobbfunksjon, f.eks. *Selger*, *Sekretær*, *Utvikler*.
- Brukere som får rollen arver alle rettighetene rollen har.
- **Fordel:** Rettigheter administreres ett sted, og endringer slår automatisk ut for alle brukere som har rollen.

Grupperoller - En grupperolle er en rolle som primært brukes til å **samle flere brukere** under samme tilgangsprofil.

- En grupperolle er en rolle som brukes til å samle flere brukere slik at de kan få de samme rettighetene. I stedet for å gi rettigheter til hver enkelt bruker, gir man rettighetene til gruppen — og alle som er medlem av gruppen arver dem automatisk.

Formål:

- Forenkle administrasjon når mange brukere skal ha like rettigheter.
- Unngå at DBA må gi rettigheter til hver enkelt bruker manuelt.

Eksempler på bruk: Opprette en grupperolle *Selger*, Gi *Selger* SELECT og INSERT på relevante tabeller, Legge brukerne *Per*, *Silje* og *Ola* inn i rollen eller Alle arver automatisk rettighetene.

Arv av rettigheter - Roller kan arve rettigheter fra andre roller, noe som gjør det mulig å bygge hierarkier.

Eksempel på hierarki:

- Rollen *Innsyn* har SELECT-rettigheter.
- Rollen *Selger* arver *Innsyn* og får i tillegg INSERT.
- En bruker som får *Selger* får dermed begge sett med rettigheter.

Dette gir fleksibilitet og reduserer administrasjonsarbeid.

Et grovmasket sikkerhetssystem

Et grovmasket sikkerhetssystem er en enkel og oversiktlig måte å håndtere tilgangskontroll i en database på. I stedet for detaljerte rettigheter per tabell eller per bruker, grupperes brukere i noen få brede kategorier. Hver kategori får et sett med faste rettigheter som dekker deres behov. Dette gjør systemet lett å administrere, men mindre fleksibelt.

Vi skiller kun mellom noen få brukertyper.

- **Administratorer:** Får alle rettigheter, også systemrettigheter som å opprette andre brukere, starte og stoppe databasen, lage tabeller osv.
- **Endre:** Sluttbrukere som både kan avlese og endre innholdet av databasen. Får rettigheter til å avlese / sette inn / oppdatere og slette data (SELECT + INSERT + UPDATE + DELETE).

- **Innsyn:** Får rettigheter til å avlese innholdet i databasen (SELECT).
- **Utviklere:** Jobber med strukturen. Får rettigheter til å lage nye tabeller, spørringer, utsnitt, og også å endre definisjonen av eksisterende objekter. Jobber vanligvis ikke med databaser i produksjon.

Her skiller vi altså ikke mellom konkrete objekter.

SQL

SQL (Structured Query Language) er et **språk som brukes til å kommunisere med databaser**. Det lar deg hente, legge inn, endre og slette data, samt definere strukturen til databasen.

Rettigheter i SQL

GRANT (Gi rettigheter) – Brukes for å gi en bruker eller rolle tilgang til et databaseobjekt, som en tabell eller et view. Du spesifiserer hvilke operasjoner som er tillatt, f.eks. SELECT eller UPDATE.

REVOKE (Fjerne rettigheter) – Brukes for å ta bort tidligere tildelte rettigheter, enten for å begrense tilgang eller rydde opp i tilganger som ikke lenger skal gjelde.

Gi videre rettigheter

- **WITH GRANT OPTION:** Mottakeren kan gi samme rettighet videre til andre.
- **WITH ADMIN OPTION:** Mottakeren kan gi systemrettigheter videre.

Eksempler:

- **GRANT SELECT ON** <table> **TO** <user>;
- **GRANT INSERT, UPDATE ON** <table> **TO** <role>;
- **REVOKE DELETE ON** <table> **FROM** <user>;
- **GRANT ALL PRIVILEGES ON** <table> **TO** <role> **WITH GRANT OPTION**;

Privilegier

Betydningen av hver bokstav

- **r** — SELECT (lese data)
- **w** — UPDATE (endre data)

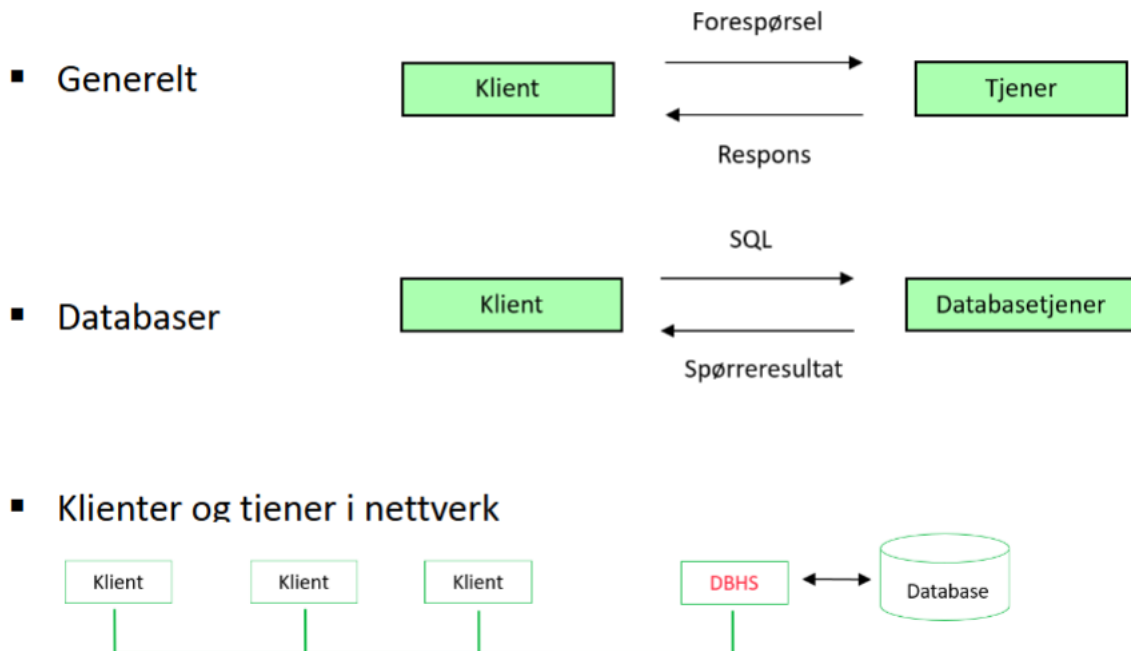
- **a** — INSERT (legge til data)
- **d** — DELETE (slette data)
- **R** — RULE (regler i databasen)
- **x** — REFERENCES (fremmednøkler)
- **t** — TRIGGER (utløse triggere)
- **X** — EXECUTE (kjøre funksjoner/prosedyrer)
- **U** — USAGE (bruke objekter som sekvenser, typer osv.)
- **C** — CREATE (opprette objekter)
- **T** — TEMPORARY (lage midlertidige tabeller)

Kombinasjoner

- **arwdRxt** — betyr *ALL PRIVILEGES* for tabeller (altså: INSERT, SELECT, UPDATE, DELETE, RULE, REFERENCES, TRIGGER)
- ***** — betyr at brukeren har **grant option**, altså lov til å gi samme rettighet videre til andre

Database i sky

Klient/tjener-arkitektur



Skyløsninger

- **IaaS (Infrastructure as a Service)** er en basistjeneste der man får tilgang på en eller flere maskiner, men må selv installere operativsystem og programvare. Eksempel: Amazon Web Services.
- **PaaS (Platform as a Service)** er en mer omfattende tjeneste enn IaaS, der man i tillegg til maskiner og operativsystem også får et ferdig utviklingsmiljø i skyen, f.eks. i form av en Linux-tjener satt opp med en MySQL database.
- **SaaS (Software as a Service)** er en ytterligere utvidet tjeneste i forhold til PaaS og kan betraktes som en måte å tilby programvare på. Eksempel: Google Apps og Office365.

Databaser i skyen

- Skydatabase (cloud database) er en database som blir lagret i skyen, og kan i utgangspunktet være basert på både IaaS, PaaS og SaaS.
- DBaaS (DataBase as a Service) er en SaaS-løsning for databaser, der man typisk både oppretter, administrerer og bruker databasen via en webapplikasjon.
- Azure SQL Database er enkelt sagt «SQL Server i skyen» og kan fås som en DBaaS-løsning.

Lagrede programmer

Lagrede rutiner er **programmer som ligger lagret inne i databasen**, og som databasen kan kjøre når du ber om det. De brukes for å automatisere oppgaver, samle logikk på ett sted og sikre at ting gjøres likt hver gang.

Lagrede rutiner

Prosedyrer (Stored Procedures)

En prosedyre er en **lagret rutine i databasen** som utfører en bestemt oppgave når den kjøres. Den kan inneholde flere **SQL-setninger**, **variabler** og **kontrollstrukturer**, og kan ta imot **parametere**.

- Dette tillater for å automatisere oppgaver, samle logikk på et sted, sikre konsistent oppførsel og øker sikkerhet ved at **brukere kan kjøre prosedyren uten direkte tilgang til tabellene**.

```
CREATE PROCEDURE oppdater_saldo(IN bruker_id INT, IN belop DECIMAL)
AS
BEGIN
    UPDATE Konto
    SET saldo = saldo + belop
    WHERE id = bruker_id;
END;
```

Funksjoner

En funksjon er en lagret rutine i databasen som **alltid returnerer en verdi**. Den brukes ofte i SELECT-spøringer og kan ta inn parametere. Funksjoner inneholder SQL-logikk, men har strengere regler enn prosedyrer og skal ikke endre data i databasen (i de fleste databaser).

```
CREATE FUNCTION beregn_moms(IN pris DECIMAL)
RETURNS DECIMAL
AS
BEGIN
    RETURN pris * 1.25;
END;
```

Parametere

Parametere er verdier som sendes inn eller ut av en prosedyre eller funksjon. De brukes for å gi rutinen informasjon den trenger, eller for å hente ut resultater.

- Det finnes tre typer parametere:

IN-parametere

Brukes for å sende verdier *inn* i en prosedyre eller funksjon. Dette tilsvarer vanlig verdioverføring i språk som Java og C. Verdien kan ikke endres av prosedyren.

OUT-parametere

Brukes for å returnere verdier *ut* fra prosedyren. Passer når man vil returnere mer enn én verdi. Variabelen får sin verdi først etter prosedyrekallet.

IN OUT-parametere

En kombinasjon av IN og OUT. Verdien sendes inn i prosedyren, **kan endres** der, og den oppdaterte verdien sendes tilbake.

Bruk av pekere (Cursors)

Cursor

- En peker som refererer en av radene i spørreresultatet.

For å behandle en databasetabell eller et spørreresultat:

- Ei løkke
 - En «rad-peker» som flyttes
 - Lese verdier i én og én rad
- ```
SELECT snr, navn,
 adresse, fdato
FROM student;
```

⇒  
cursor

| snr | navn | adresse  | fdato    |
|-----|------|----------|----------|
| 1   | EVA  | AVEIEN 1 | 01.02.97 |
| 2   | OLA  | BVEIEN 2 | 10.02.03 |
| 5   | KARI | AVEIEN 4 | 02.10.99 |
| 7   | PER  |          | 02.10.92 |
| 9   | GURI | CVEIEN 5 | 15.11.97 |
|     |      |          |          |

Eksempel:

```

DO $$
DECLARE
 cur CURSOR FOR
 SELECT id, navn
 FROM <table>;

 v_id <table>.id%TYPE;
 v_navn <table>.navn%TYPE;
BEGIN
 OPEN cur;

 LOOP
 FETCH cur INTO v_id, v_navn;
 EXIT WHEN NOT FOUND;

 RAISE NOTICE 'ID: %, Navn: %', v_id, v_navn;
 END LOOP;

 CLOSE cur;
END;
$$;

```

## Triggere

En databasetrigger er prosedyrekode som automatisk kjøres som respons på bestemte hendelser (som INSERT, UPDATE, DELETE) på en bestemt tabell [...] i en database. Triggeren brukes hovedsakelig til å opprettholde integriteten til informasjonen i databasen.

- Triggeren «**avfyses**» når hendelsen skjer, uten at klienten trenger å gjøre noe.

**Formål:** sikre dataintegritet, automatisere oppgaver, loggføre endringer, håndheve forretningsregler.

## Hvordan lage en trigger?

```

CREATE TRIGGER triggernavn
 [BEFORE | AFTER]
 [INSERT | UPDATE | DELETE] ON tabellnavn
 [FOR EACH ROW]
BEGIN
 setninger
END

```

Funksjon:

Knytt funksjonen til en trigger:

```

CREATE TRIGGER trig_navn
BEFORE UPDATE ON tabell
FOR EACH ROW
EXECUTE FUNCTION trig_fn();

```

```
CREATE OR REPLACE FUNCTION trig_fn()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
 -- logikk
 RETURN NEW; -- eller OLD ved DELETE
END;
$$;
```

## Hvordan finne triggere i systemkatalogen

Du kan hente metadata om triggere via:

```
SELECT tgname, relname
FROM pg_trigger
JOIN pg_class ON pg_class.oid = tgrelid;
```

### *Old vs. New*

OLD og NEW er innebygde «postvariabler» som inneholder radverdier:

| Hendelse | OLD                  | NEW                   |
|----------|----------------------|-----------------------|
| INSERT   | NULL                 | Nye verdier           |
| UPDATE   | Verdier før endring  | Verdier etter endring |
| DELETE   | Verdier før sletting | NULL                  |

- **VIKTIG:** OLD og NEW kan bare brukes i **rad-triggere**.

*Before and after*

## Rad-triggere

|        | BEFORE                             | AFTER                               |
|--------|------------------------------------|-------------------------------------|
| INSERT | Ny rad er ikke satt inn ennå       | Ny rad er satt inn i tabellen       |
| UPDATE | Raden er ikke oppdatert i tabellen | NEW-verdier er oppdatert i tabellen |
| DELETE | Raden er ikke slettet ennå         | Raden er slettet                    |

## Setnings-triggere

|        | BEFORE                        | AFTER                                                    |
|--------|-------------------------------|----------------------------------------------------------|
| INSERT | Ingen rader er satt inn ennå  | Alle nye rader er satt inn i tabellen                    |
| UPDATE | Ingen rader er oppdatert ennå | Alle berørte rader er oppdatert i tabellen               |
| DELETE | Ingen rader er slettet ennå   | Alle rader som berøres av DELETE er slettet fra tabellen |

### BEFORE-triggere

- Kjøres **før** databasen oppdateres.
- Kan endre **NEW**-verdier før de lagres.
- Brukes til:
  - Automatisk formatering (f.eks. UPPERCASE)
  - Sette datostempel
  - Validere forretningsregler (f.eks. pris < 10000)

### AFTER-triggere

- Kjøres **etter** at databasen er oppdatert.
- Kan ikke endre NEW-verdier.
- Brukes til:
  - Loggføring
  - Oppdatere andre tabeller
  - Oppdatere avledede data

*Typer triggere*

### 1. Rad-triggere (FOR EACH ROW)

- Kjøres **én gang per rad** som blir berørt.
- Brukes når logikken avhenger av verdiene i hver enkelt rad.

- Gir tilgang til **OLD** og **NEW**.

Eksempelbruk:

- Automatisk formatering av tekst
- Oppdatere datostempel
- Loggføring av endringer
- Oppdatere andre tabeller basert på radens innhold

## 2. Setnings-triggere

- Kjøres **én gang per SQL-setning**, uansett hvor mange rader som endres.
- Brukes til:
  - Aksesskontroll
  - Overordnet validering
  - Oppgaver som ikke avhenger av individuelle radverdier

### *Bruksområder for triggere*

#### **BEFORE triggere – fyres før oppdatering i basen**

- Automatisk formatering av verdier
- Automatisk datostempling av rader
- Kontroll med forretningsregler
- Finmasket tilgangskontroll

#### **AFTER triggere - fyres etter oppdatering i basen**

- Automatiske oppdateringer i andre tabeller
- Loggføring av oppdateringer
- Oppdatere avledete data i andre tabeller

**Brukes for operasjoner som skal gjøres «uten unntak»!**

### *Vanlige bruksområder for triggere*

**1. Automatisk formatering** - Eksempel: gjøre fornavn til store bokstaver før lagring.

**2. Automatisk datostempling** - Oppdatere endret dato ved hver endring.

**3. Forretningsregler** - Eksempel: hindre at pris > 10000.

**4. Loggføring** - Lagre gamle og nye verdier i en loggtabell ved endringer.

**5. Automatiske følgeoppdateringer** - Eksempel: oppdatere saldo når en ny transaksjon settes inn.

**6. Tilgangskontroll** - Setnings-triggere kan brukes til å hindre uønskede operasjoner.

## Forretningsregler

Forretningsregler i databaser handler om **regler som sikrer at data alltid følger kravene virksomheten har satt**, uansett hvem som bruker systemet eller hvordan data legges inn.

- Forretningsregler er **regler som bestemmer hva som er lov og ikke lov i databasen**, basert på krav fra virksomheten. De sørger for at data er gyldige, konsistente og følger bedriftens logikk.

### *Hvordan forretningsregler implementeres*

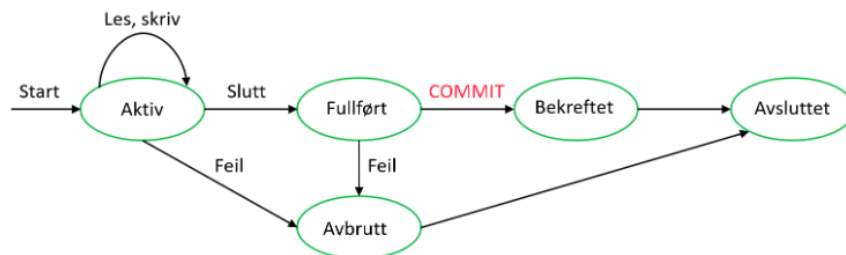
1. **CHECK-constraints** Enkle regler direkte i tabellen.
2. **Triggere (BEFORE)** Mer avanserte regler som krever logikk, f.eks. feilmeldinger, formatering eller beregninger.
3. **Lagrede rutiner** Når regler må kjøres som del av større prosesser.

## Transaksjoner og gjenoppretning

En transaksjon er en **samling logisk operasjon** mot databasen. Enten lykkes hele transaksjonen, eller så utføres ingen endringer. Dette sikrer at databasen alltid holder seg i en konsistent og korrekt tilstand.

Transaksjoner begynner med **BEGIN**, og avsluttes med:

- **COMMIT**: bekreft, endringene utføres
- **ROLLBACK**: angre, endringene tilbakestilles

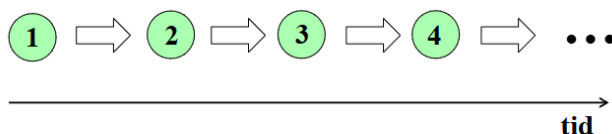


**AUTO-COMMIT** betyr at databasen automatisk gjør hver SQL-operasjon permanent med én gang den kjører. Det betyr at du ikke trenger å skrive COMMIT selv etter hver INSERT, UPDATE eller DELETE.

- Auto-commit er standard, og kan skrus av med **SET autocommit = 0;**

**Tilstand** - En tilstand er innholdet i databasen på et bestemt tidspunkt.

- **En transaksjon bringer databasen fra én tilstand til en annen.**
- Hvis vi tenker oss at kun én transaksjon blir utført av gangen, kan livsløpet til en database visualiseres slik:



## Transaksjonsloggen

Transaksjonsloggen er en **sekvensiell logg** der databasen skriver ned alle endringer *før* de utføres på selve databasen.

- Transaksjonsloggen er en **kritisk del av et databasesystem** og brukes for å sikre at databasen kan **håndtere feil, gjenopprettes og opprettholde ACID-egenskapene**.

| TransID | Table          | RowID  | Attribute | Before | After |
|---------|----------------|--------|-----------|--------|-------|
| 101     | *** BEGINTRANS |        |           |        |       |
| 101     | VARE           | 102375 | ANTALL    | 112    | 113   |
| 101     | VARE           | 102542 | ANTALL    | 56     | 66    |
| 101     | *** COMMIT     |        |           |        |       |

## ACID-egenskapene

ACID-egenskapene beskriver **kriterier en transaksjon i en database må oppfylle** for at data skal være **trygge, konsistente og pålitelige**.

1. **Atomicity:** Alle eller ingen av deloperasjonene i en transaksjon må fullføres.
2. **Consistency:** En transaksjon fører databasen fra en lovlig tilstand til en annen lovlig tilstand.
3. **Isolation:** Effekt av transaksjoner under utførelse skal ikke kunne observeres av andre transaksjoner.
4. **Durability:** Effekt av fullførte (committed) transaksjoner lagres i databasen og skal ikke gå tapt på grunn av feil.

## Isolasjonsnivåer

Isolasjonsnivåer beskriver hvor mye transaksjoner får lov til å påvirke hverandre mens de kjører, og hvilke typer samtidighetsproblemer som kan oppstå. De fire klassiske nivåene er definert i SQL-standardens.

| Isolasjonsnivå   | Fantomer | Ikke-repeterbar lesing | Usikker lesing |
|------------------|----------|------------------------|----------------|
| SERIALIZABLE     | Nei      | Nei                    | Nei            |
| REPEATABLE READ  | Ja       | Nei                    | Nei            |
| READ COMMITTED   | Ja       | Ja                     | Nei            |
| READ UNCOMMITTED | Ja       | Ja                     | Ja             |

- **Serializable** gir full beskyttelse mot alle anomalier og oppfører seg som om transaksjoner kjører én etter én.
- **Repeatable Read** hindrer at verdier endres mellom to lesinger, men nye rader kan fortsatt dukke opp (phantoms).
- **Read Committed** hindrer dirty reads, men både verdier og rader kan endre seg mellom lesinger.
- **Read Uncommitted** gir ingen beskyttelse og tillater alle anomalier.

## Samtidighetskontroll

Samtidighetskontroll handler om **hvordan databasen sørger for at flere brukere og transaksjoner kan jobbe samtidig uten å ødelegge hverandres data**.

- Det er et sett med mekanismer som beskytter databasen mot feil som oppstår når to eller flere transaksjoner prøver å lese eller skrive de samme dataene samtidig.
- **Samtidigheidskontroll sørger for at:** *To transaksjoner ikke overskriver hverandre, Lesing og skriving skjer i riktig rekkefølge, Databasen holder seg konsistent selv om mange jobber parallelt og at ACID-egenskapene (spesielt Isolation) blir oppfylt.*

Uten god samtidigheidskontroll kan samtidige transaksjoner skape alvorlige problemer:

- **Lost update (tapt oppdatering)**  
Én transaksjon overskriver en annen sin oppdatering.
- **Dirty read (angret oppdatering)**  
En transaksjon leser data som en annen transaksjon senere ruller tilbake.
- **Incorrect summary (inkonsistent analyse)**  
En transaksjon leser både gamle og nye verdier fordi en annen transaksjon oppdaterer samtidig.

### Løsningen på samtidigheidsproblemene: **Låser**

En lås (lock) er en «ventemekanisme» som gjør at **transaksjoner må vente på hverandre.**

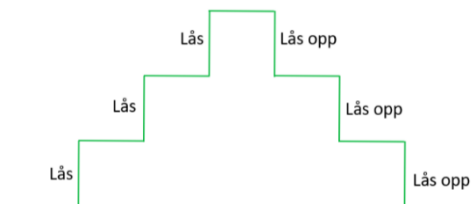
- En transaksjon **kan låse** større eller mindre **deler**:
  - hele databasen
  - en tabell
  - en rad i en tabell
  - en «celle» i en rad i en tabell

Vi skiller mellom: **skrivelåser (exclusive locks)** og **leselåser (shared locks)**.

### Tofaselåsing

Tofaselåsing betyr at en transaksjon først bare kan ta låser (vekstfase) og deretter bare kan slippe låser (krympfase). Denne strukturen gjør at samtidige transaksjoner alltid får et serialiserbart og korrekt resultat.

En transaksjon følger reglene for tofaselåsing hvis alle låseoperasjoner gjøres før første frigivelse (opplåsing).



- Det vil si, **ingen låsing etter første opplåsing!**

*Vranglås* – problem som kan oppstå

Vranglås (**deadlock**) er en situasjon der to eller flere transaksjoner blokkerer hverandre fordi hver av dem holder en lås den andre trenger, slik at ingen av dem kan fortsette. Ingen kommer videre uten at databasen oppdager problemet og avbryter én av transaksjonene.

Forhindre vranglås: **Alle transaksjoner gis et unikt tidsstempel.**

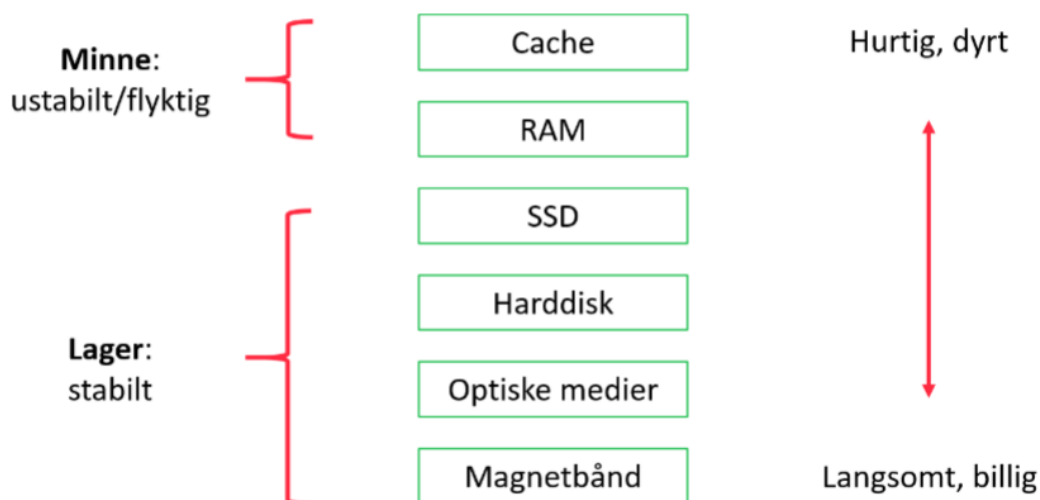
- Hvis en transaksjon vil måtte vente på en eldre transaksjon blir den avbrutt. Det betyr at yngre transaksjoner aldri vil vente på eldre og dermed kan det ikke oppstå sykler.

*Serialiserbarhet*

Serialiserbarhet betyr at **resultatet** av flere transaksjoner som kjører samtidig skal være det **samme** som om de hadde **kjørt én etter én** i en eller **annen rekkefølge**. Det er kjernen i hvorfor vi har samtidighetskontroll.

Filer, indekser og dataformer

## Lagringsmedier



- **Cache** – Svært raskt, lite minne som brukes for å mellomlagre data prosessoren trenger umiddelbart.
- **RAM (arbeidsminne)** – Raskt og flyktig minne som lagrer data midlertidig mens programmer kjører.
- **SSD** – Hurtig og stabil lagring uten bevegelige deler; brukes som hovedlagring i moderne PC-er.
- **Harddisk (HDD)** – Mekanisk lagring med større kapasitet, men tregere enn SSD.
- **Optiske medier** – CD/DVD/Blu-ray; brukes til lagring og distribusjon av data, relativt tregt.
- **Magnetbånd** – Svært billig og langsomt, brukes mest til sikkerhetskopiering i store datasentre.

### Terminologi

Samme konsept i databaser har ulike navn avhengig av hvilket nivå eller modell du snakker om – SQL, relasjonsmodellen, ER-modellen eller de fysiske filene på lagringsmediet.

| SQL-språk      | Relasjonsmodellen | ER-modellen       | Fysiske filer | Hva betyr det?             |
|----------------|-------------------|-------------------|---------------|----------------------------|
| <b>Tabell</b>  | Relasjon          | Entitetstype      | Fil           | Det logiske datasettet     |
| <b>Rad</b>     | Tuppel            | Entitetsforekomst | Post          | Ett konkret objekt/instans |
| <b>Kolonne</b> | Attributt         | Attributt         | Felt          | En egenskap ved objektet   |

- SQL, relasjonsmodellen, ER-modellen og fysiske filer beskriver *samme data*, men på ulike nivåer.
- Derfor brukes forskjellige ord, selv om de peker på omtrent de samme tingene.
- **Eksempel:** I SQL sier du *tabell*, i relasjonsmodellen heter det *relasjon*, i ER-modellen er det *entitetstype*, og fysisk på disken er det bare en *fil*.

## Blokker

En blokk er den **minste enheten som flyttes mellom minne og lagringsmedium**.

Vanlig blokkstørrelse er 4 KB (4096 byte). Én blokk inneholder som regel mange rader fra en tabell. Databasesystemet (DBHS) holder styr på hvilke data som ligger i hvilke blokker.

## Fil delt inn i blokker

|          | KNr | Fornavn | Etternavn | Telefon     |           |
|----------|-----|---------|-----------|-------------|-----------|
| Post 1   | 1   | Elias   | Hansen    | 99 88 77 66 | } Blokk 1 |
| Post 2   | 2   | Hulda   | Akselsen  | 31 45 88 21 |           |
| Post 3   | 5   | John    | Boine     | 23 94 53 18 |           |
|          | ... | ...     | ...       | ...         |           |
| Post 58  | 63  | Mari    | Gygre     | 55 66 77 88 | } Blokk 2 |
| Post 59  | 64  | Michael | Svensen   | 19 82 37 64 |           |
| Post 60  | 69  | Robert  | Romman    | 91 28 73 46 |           |
| Post 61  | 72  | Laura   | Eika      | 64 37 82 19 |           |
|          | ... | ...     | ...       | ...         |           |
| Post 116 | 129 | Andreas | Karlsen   | 51 15 83 38 |           |
|          | ... | ...     | ...       | ...         |           |

- ❑ En fil blir lagret i et antall blokker.
- ❑ Det settes gjerne av litt ledig plass i hver blokk.
  - Gjør det lettere å sette inn / slette rader (poster).
  - **Fyllingsgraden** (f.eks. 80%) kan styres i noen DBHS.

## Heap-filer

En **heap-fil** er en tabell der radene lagres **uten noen bestemt rekkefølge**. Nye rader legges bare til «øverst i bunken», noe som gjør innsetting raskt, men søk tregere fordi databasen må lete gjennom hele tabellen.

# Filstrukturer

❑ **Ikke-sorterte filer** (Heap-filer): «Tilfeldig» rekkefølge.

➤ Nye poster legges inn til slutt.

|    |    |   |    |    |    |    |  |  |  |
|----|----|---|----|----|----|----|--|--|--|
| 87 | 27 | 7 | 16 | 41 | 19 | 50 |  |  |  |
|----|----|---|----|----|----|----|--|--|--|

Hver boks symboliserer en post.

❑ **Sorterte filer** (sekvensielle filer): Sortert, f.eks. med hensyn på VNr.

|   |    |    |    |    |    |    |  |  |  |
|---|----|----|----|----|----|----|--|--|--|
| 7 | 16 | 19 | 27 | 41 | 50 | 87 |  |  |  |
|---|----|----|----|----|----|----|--|--|--|

Et antall poster i hver blokk

## Indekser

- En indeks er en datastruktur som gjør søk mye raskere.
- Man kan ha flere indekser på samme tabell, men de krever ekstra lagringsplass og må oppdateres ved endringer.
- En indeks fungerer som en registerliste i en bok: sorterte søkeord med referanser.
- I databaser peker referansene til diskadresser.
- Søk går raskere fordi indeksen er mindre og sortert, i motsetning til hele datafilen.

| Nøkkel   | Adresse | KNr | Fornavn | Etternavn | Telefon     |
|----------|---------|-----|---------|-----------|-------------|
| Akselsen | →       | 1   | Elias   | Hansen    | 99 88 77 66 |
| Boine    | →       | 2   | Hulda   | Akselsen  | 31 45 88 21 |
| Eika     | →       | 5   | John    | Boine     | 23 94 53 18 |
| Gygre    | →       | 63  | Mari    | Gygre     | 55 66 77 88 |
| Hansen   | →       | 64  | Michael | Svensen   | 19 82 37 64 |
| Karlsen  | →       | 69  | Robert  | Romman    | 91 28 73 46 |
| Romman   | →       | 72  | Laura   | Eika      | 64 37 82 19 |
| Svensen  | →       | 129 | Andreas | Karlsen   | 51 15 83 38 |

### Indekser med SQL

- Den enkleste varianten:

```
CREATE INDEX VarenavnIdx
ON Vare (Betegnelse)
```

- Indekser kan hindre repetisjoner:

```
CREATE UNIQUE INDEX EtternavnIdx
ON Ansatt (Etternavn)
```

- Indekser kan være sammensatte:

```
CREATE INDEX NavnIdx
ON Ansatt (Etternavn, Fornavn)
```

Får her indeks på etternavn «på kjøpet» !

### Tette og ikke-tette indekser

- Tett indeks (dense): Har én indeksoppføring for hver rad i datafilen.
- Ikke-tett indeks (sparse): Har én indeksoppføring per blokk, ikke per rad.
- Ikke-tette indekser er mindre fordi én blokk inneholder mange rader.
- Det kan kun finnes én ikke-tett indeks per datafil, fordi filen bare kan være sortert på én måte (f.eks. kundenummer).

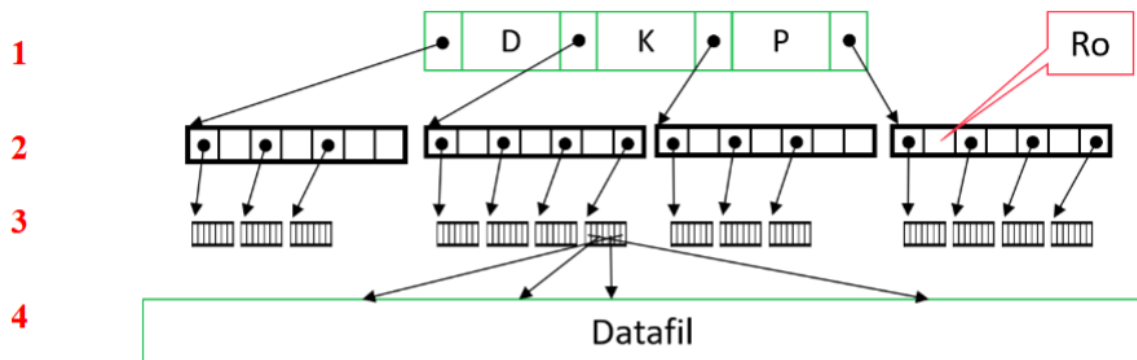
### B<sup>+</sup>-trær

- Brukes som flernivå-indekser når en indeks blir for stor til å søkes effektivt i ett nivå.

- Indeksene organiseres som et tre, der søk starter i roten og går nedover til bladene.
- Indre noder kan være ikke-tette indekser, fordi nivået under allerede er sortert.
- Antall diskaksesser ved søk = treets dybde.
- B<sup>+</sup>-trær er balanserte, så **alle søk tar omtrent like mange operasjoner**.
- Bladnoder peker videre til datafilen, der selve posten ligger.

Antall lese-  
operasjoner:

### B<sup>+</sup>-trær



Anta vi skal finne post der Etternavn = «Roheim».

- Les inn rot-blokken.
- Søk i rot-blokken: Her skal vi følge pekeren helt til høyre.
- Gjenta søk på mellomliggende nivåer ...
- ... Les inn riktig blokk fra datafilen.

#### Valg av indeks

- **Bør indekseres:** kolonner man ofte søker i eller sorterer på, primærnøkler, fremmednøkler og kolonner som brukes i koblinger (JOINS).
- **Bør ikke/kan ikke indekseres:** kolonner med få ulike verdier (f.eks. Ja/Nei), samt store datafelt som bilder, video, lyd og lange dokumenter.
- **Ulemper:** indekser bruker lagringsplass og må oppdateres hver gang tabellen endres.

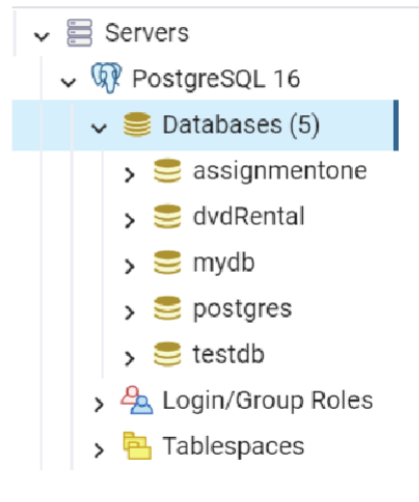
## Datatyper

- **Tekst:** Bruk CHAR for faste lengder (f.eks. fødselsnummer), VARCHAR ellers. Sett en fornuftig maksgrense.
- **Tall:** Velg så liten datatype som mulig. Bestem om du trenger heltall eller desimaltall (f.eks. kronebeløp).
- **Tekst eller tall?** Hvis du skal *regne* med verdiene → bruk tall.
- **Tid:** Velg datatype basert på om du trenger dato, klokkeslett eller begge deler.

## PostgreSQL Storage Structure (Fra kapittel 9)

### PostgreSQL Storage Structure

- ❑ Database cluster is a collection of databases that is managed by a single instance of a running database server.
- ❑ Before you can do anything, you must initialize a database storage area on disk.
- ❑ After initialization, a database cluster will contain a database named “postgres”, which is meant as a default database for use by utilities, users and third-party applications.



### PostgreSQL Storage Structure

- ❑ Query the structure of “dvdRental” database:

Query

Query History

1

SELECT relname, oid, relpages, reltuples FROM pg\_class;

Data Output

Messages

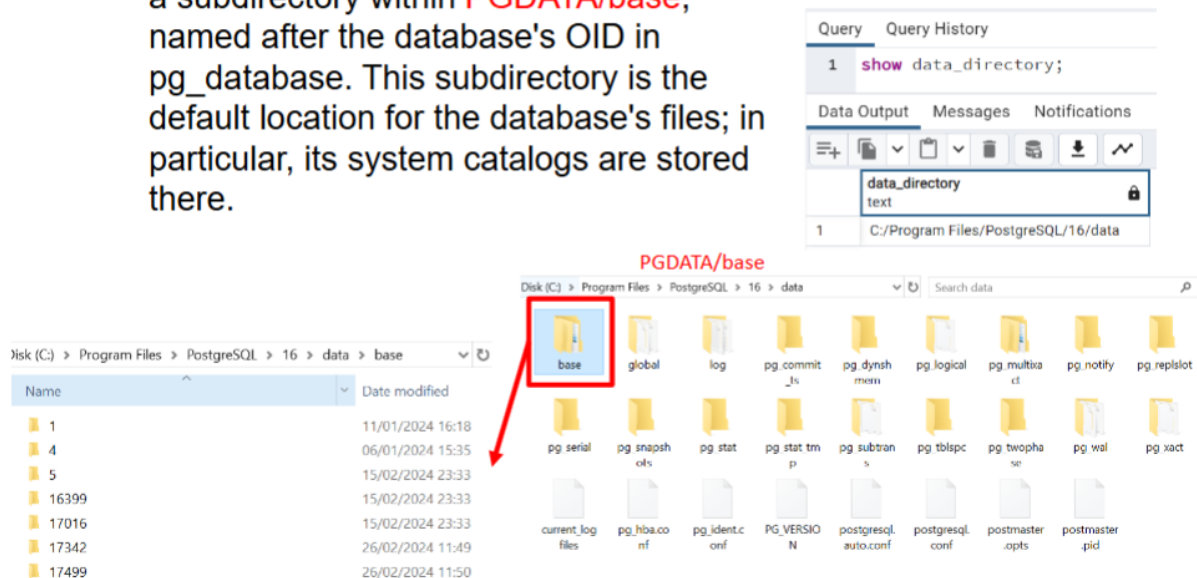
Notifications



|    | relname<br>name | oid<br>[PK] oid | relpages<br>integer | reltuples<br>real |
|----|-----------------|-----------------|---------------------|-------------------|
| 29 | actor_pkey      | 16555           | 2                   | 200               |
| 30 | address_pkey    | 16557           | 4                   | 603               |
| 31 | category_pkey   | 16559           | 2                   | 16                |
| 32 | address         | 16470           | 8                   | 603               |
| 33 | category        | 16439           | 1                   | 16                |
| 34 | city            | 16476           | 5                   | 600               |
| 35 | country         | 16482           | 1                   | 109               |
| 36 | customer        | 16423           | 9                   | 599               |
| 37 | film_actor      | 16456           | 30                  | 5462              |
| 38 | film_category   | 16460           | 6                   | 1000              |
| 39 | inventory       | 16498           | 25                  | 4581              |
| 40 | language        | 16504           | 1                   | 6                 |
| 41 | rental          | 16520           | 150                 | 16044             |

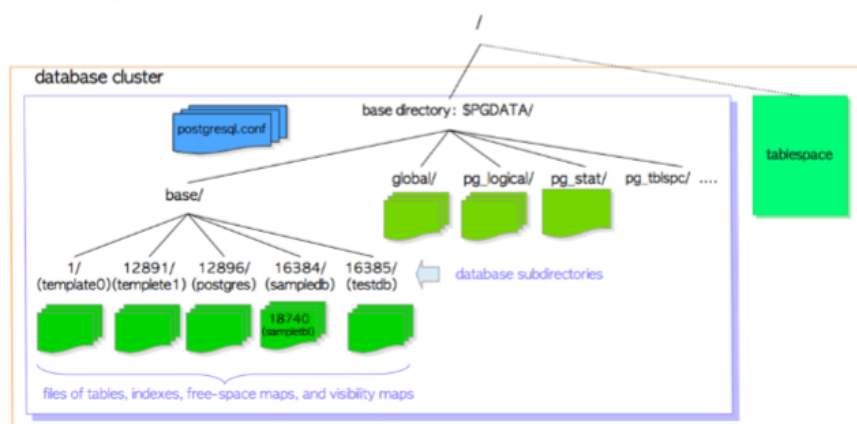
## PostgreSQL Storage Structure

- For each database in the cluster there is a subdirectory within **PGDATA/base**, named after the database's OID in `pg_database`. This subdirectory is the default location for the database's files; in particular, its system catalogs are stored there.



## PostgreSQL Storage Structure

- ❑ In file system terms, the configuration and data files used by a database cluster are stored together within the cluster's data directory, commonly called PGDATA.
- ❑ The PGDATA directory contains several subdirectories and control files.
- ❑ The cluster configuration files postgresql.conf, pg\_hba.conf, and pg\_ident.conf are traditionally stored in PGDATA, although it is possible to place them elsewhere.



# PostgreSQL Storage Structure

- ❑ The OIDs listed in the query result represent the actual names for the databases in the physical file system. So, the database “dvdRental” is represented by a folder named “16399” and not “dvdRental”. So the path of “dvdRental” database can be: “C:\Program Files\PostgreSQL\16\data\base\16399”, where all files concerning this database are stored.

disk (C:) > Program Files > PostgreSQL > 16 > data > base

| Name  | Date modified    |
|-------|------------------|
| 1     | 11/01/2024 16:18 |
| 4     | 06/01/2024 15:35 |
| 5     | 15/02/2024 23:33 |
| 16399 | 15/02/2024 23:33 |
| 17016 | 15/02/2024 23:33 |
| 17342 | 26/02/2024 11:49 |
| 17499 | 26/02/2024 11:50 |

Query Query History

1 SELECT oid, datname FROM pg\_database;

Data Output Messages Notifications

|   | oid<br>[PK] oid | datname<br>name |
|---|-----------------|-----------------|
| 1 | 5               | postgres        |
| 2 | 1               | template1       |
| 3 | 4               | template0       |
| 4 | 16399           | dvdRental       |
| 5 | 17016           | assignmentone   |
| 6 | 17342           | testdb          |
| 7 | 17499           | mydb            |

Query Query History

1 SELECT oid, datname FROM pg\_database WHERE datname = 'dvdRental';

Data Output Messages Notifications

You can also use this command for a specific database

|   | oid<br>[PK] oid | datname<br>name |
|---|-----------------|-----------------|
| 1 | 16399           | dvdRental       |

# PostgreSQL Storage Structure

- ❑ The same goes for the tables created in each database, created tables and objects are stored in the database’s folder “16399” by their OID and not their textual name. PostgreSQL stores each table in the database in a separate file (a heap file), this file is named after the table’s filenode number. This filenode number often matches the table’s OID.

Program Files > PostgreSQL > 16 > data > base > 16399

Search 16399

| Name      | Date modified    | Type |
|-----------|------------------|------|
| 14842     | 15/01/2024 06:51 | File |
| 14842_fsm | 15/01/2024 06:51 | File |
| 14842_vm  | 15/01/2024 06:51 | File |
| 14845     | 15/01/2024 06:45 | File |
| 14846     | 15/01/2024 06:51 | File |
| 16422     | 15/01/2024 06:55 | File |
| 16423     | 15/01/2024 06:55 | File |
| 16423_fsm | 15/01/2024 06:55 | File |

What is fsm?

Query Query History

1 select pg\_relation\_filepath('customer');

Data Output Messages Notifications

|   | pg_relation_filepath<br>text |
|---|------------------------------|
| 1 | base/16399/16423             |

The filenode number for «customer» table is 16423

Query Query History

1 SELECT pg\_relation\_filenode('customer');

Data Output Messages Notifications

|   | pg_relation_filenode<br>oid |
|---|-----------------------------|
| 1 | 16423                       |

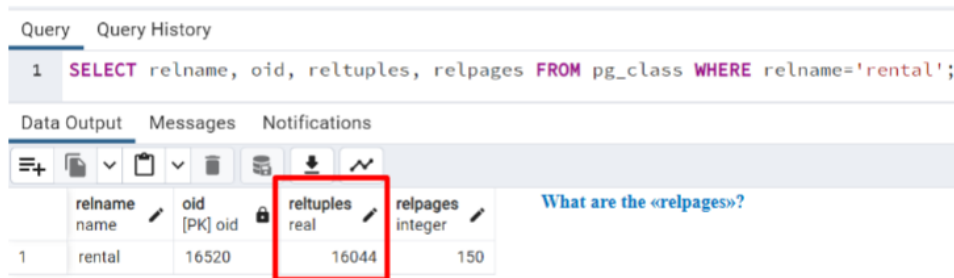
To retrieve the filenode number for «customer» table without the file path

Or you could use this query

1 SELECT oid FROM pg\_class WHERE relname = 'customer';

# PostgreSQL Storage Structure

- ❑ Query the number of pages and tuples in table “rental” (the biggest table in “dvdRental” database:



The screenshot shows a PostgreSQL query window with the following SQL query: `SELECT relname, oid, reltuples, relpages FROM pg_class WHERE relname='rental';` The results are displayed in a table with columns: relname, oid, reltuples, and relpages. The 'rental' table is listed with oid 16520, 16044 tuples, and 150 pages. The 'reltuples' column is highlighted with a red box.

| relname | oid   | reltuples | relpages |
|---------|-------|-----------|----------|
| rental  | 16520 | 16044     | 150      |

## Hierarkiske dataformater

Hierarkiske dataormer er dataformater som organiserer **data i et tre- eller nodebasert hierarki**, typisk brukt i XML og enkelte NoSQL-databaser.

### JSON: JavaScript Object Notation

JSON er et tekstbasert dataformat brukt til å representere komplekse og semistrukturerte data. Det er spesielt vanlig i web-utvikling og API-er, og brukes mye som utvekslingsformat mellom systemer.

#### JSON bygger på noen få grunnleggende elementer:

- **Enkle verdier:** tall, tekst, true, false, null  
«Grunnleggende verdier: tall, tekst, true/false, null»
- **Objekter:** nøkkel/verdi-par  
Eksempel: { "tittel": "Salmer fra kjøkkenet", "år": 2003 }
- **Arrays:** lister av verdier  
Eksempel: [1, 3, 5, 7, 9]

Objekter og arrays kan kombineres for å lage **komplekse datastrukturer**.

```
{
 "KNr":5321,
 "Navn":{
 "Fornavn":"Sivert",
 "Etternavn":"Ose"
 },
 "Adresse":"Harald Halvorsens vei 17",
 "PostNr":"0208",
 "Poststed":"Oslo",
 "Ordrer":[
 {
 "OrdreNr":20571,
 "OrdreDato":"2024-09-24",
 "Ordrelinjer":{
 "VNr":"49921",
 "Betegnelse":"Lezlo leire, økonomipakke",
 "PrisPrEnhet":514.00,
 "Antall":1
 },
 "VNr":"64509",
 "Betegnelse":"Kongelilje, 10 stk.",
 "PrisPrEnhet":204.00,
 "Antall":5
 }
]
}
```

## Hvorfor JSON brukes

- Lett å lese for både mennesker og maskiner
- Passer godt for **semi-strukturerte data**
- Støttes av mange språk, spesielt JavaScript
- Vanlig format i API-er og åpne datasett

## JSON i databaser

En tabell kan representeres som:

- én array med rader
- hver rad som et JSON-objekt  
«Hvert tabellrad blir til et JSON-objekt ... settes sammen i en JSON-array.»

JSON gir også mulighet for **embedding**, der relaterte data legges inni hverandre (f.eks. kunde → ordre → ordrelinjer).

## XML

XML (eXtensible Markup Language) er et tekstbasert, fleksibelt og utvidbart format for å representere data i en trestruktur. Det er et metaspråk, altså et språk for å definere nye spark.

- **Utvidbart:** ingen forhåndsdefinerte elementer (i motsetning til HTML).  
«XML er utvidbart (extensible)»
- **Selvbeskrivende:** elementnavn forteller hva data betyr.
- **Strukturert som et tre:** foreldre, barn, søsken.
- **Brukes i mange bransjer:** f.eks. GML, ebXML, CML.

## XML i databaser

XML kan:

- lagres som **CLOB** eller **XMLType**
- spørringer kan gjøres med **XPath** eller **XQuery**
- genereres fra tabelldata

Eksempel: `SELECT xpath('/breakfast_menu/food[name="French Toast"]', DATA)`

## XPath

XPath er et **spørrespråk for XML** som brukes til å navigere i og hente ut deler av et XML-dokument. «*XPath er en notasjon for å referere til deler av et XML-dokument*». Det fungerer litt som filstier i et operativsystem — du peker deg frem til riktig element i trestrukturen.

Hvorfor brukes XPath?

- For å hente ut spesifikke elementer, attributter eller tekst
- For å lage regler som «skriver om» XML-dokumenter
- For å bruke XML i databaser (f.eks. PostgreSQL)
- For å bygge mer avanserte spørringer i XQuery

### Grunnleggende syntaks for XPath

XPath bruker **stier** for å navigere i XML-treet:

| XPath-uttrykk         | Betydning                                |
|-----------------------|------------------------------------------|
| /                     | Fra rotnivå                              |
| //                    | Finn elementer uansett hvor i dokumentet |
| book/title            | title som ligger inni book               |
| @id                   | Attributtet id                           |
| //book[1]             | Første book-element                      |
| //book[author="Kari"] | book der author har teksten "Kari"       |

### XML og databaser

XML kan lagres, behandles og brukes i databaser, samt hvorfor XML er nyttig som utvekslingsformat mellom systemer.

### Hvorfor lagre XML i databaser?

XML brukes ofte når data er:

- **semistrukturerte**
- **dokumentbaserte**
- **varierende i struktur**
- **skal utveksles mellom systemer**

Databaser kan lagre XML for å:

- hente ut deler av dokumentet
- validere struktur
- kombinere XML-data med relasjonelle data
- produsere XML som output

### Hvordan lagres XML i databaser?

#### 1. Som tekst (CLOB)

- XML lagres som ren tekst
- Ingen spesialisert XML-funksjonalitet
- Billig og enkelt, men lite funksjonelt

#### 2. Som XMLType (eller tilsvarende datatype)

- Databasen forstår XML-strukturen
- Mulighet for validering, indeksing og spørringer
- PostgreSQL bruker XML-datatype og funksjoner som XMLPARSE

## NoSQL

NoSQL er en type **ikke-relasjonell database** som lagrer data uten faste tabeller og skjemaer, og som er laget for **fleksibilitet, skalerbarhet og høy ytelse**, spesielt ved store datamengder.

- Relasjonsdatabaser er laget for **strukturerte data i tabellform**, og fungerer dårlig når dataene blir mer komplekse, hierarkiske eller uforutsigbare.

**Typer data som ikke passer godt i tabeller (relasjonelle databaser):**

- Geografiske informasjonssystemer (GIS)
- Tekniske tegninger
- Multimedia (bilder, video, lyd)
- Semistrukturerte dokumenter (XML, JSON)
- Sosiale medier-data
- Stordata fra IoT (Internet of Things)

Behovet for å håndtere slike datastrukturer førte til:

- **objektrelasjonelle databaser** (relasjonsdatabaser + objektorienterte mekanismer)
- **NoSQL-databaser** (dokumentdatabaser, grafdatabaser, key/value, osv.)

## Data typer

### *Data typer i PostgreSQL*

Støtte for SQL og standard datatyper. Dessuten:

- Brukerdefinerte datatyper
- Domenetyper
- Array-typer
- Oppramstyper
- Intervalltyper
- Geometrityper
- XML- og JSON-datatyper

### *Brukerdefinerte datatyper (User-defined types)*

Du kan lage helt egne datatyper som passer domenet ditt. Dette gjør det mulig å modellere komplekse datastrukturer direkte i databasen.

Eksempler:

- Egne strukturer (f.eks. koordinater, adresser)
- Arv mellom typer (subklasser)

### Domene typer (Domain types)

En domenetype er en **eksisterende datatype + en regel (CHECK)**.

Eksempel:

```
CREATE DOMAIN AlderType AS SMALLINT
CHECK (VALUE BETWEEN 0 AND 150);
```

Brukes for å sikre **datakvalitet** på tvers av tabeller.

### Arrays og oppramstyper (ENUM)

En array brukes for å ta vare på en liste med verdier av samme datatype, f.eks. en liste med telefonnumre. Eksempel på kolonnedefinisjon: Tlf TEXT[]

En ENUM-type lar deg definere en **fast liste med lovlige verdier**.

Eksempel: `CREATE TYPE StatusType AS ENUM ('aktiv', 'permisjon', 'sluttet');`

Gir tryggere data enn fritekst.

## NoSQL og Big Data

**NoSQL** betyr Not Only SQL

NoSQL er nye typer databaser som passer bedre for:

- komplekse datastrukturer
- semistrukturerte data (f.eks. JSON)
- store, distribuerte systemer

.. fordi de har **slappere krav til transaksjoner** enn vanlige relasjonsdatabaser

### Big Data

Handler om **svære og komplekse datasett** som er vanskelige å lagre og behandle

- Kjennetegnes av de 5 V-ene:
- **Volume** (mengde)
- **Velocity** (hastighet)
- **Variety** (mangfold)

- **Veracity** (kvalitet/usikkerhet)
- **Value** (nytteverdi)

## Nøkkel/verdi-databaser (key/value)

Data er organisert ved en samling av nøkkel/verdi-par, som f.eks. Fornavn='Ola'.

- Velegnet for applikasjoner som gjør mange, enkle oppslag basert på nøkler.

| Nøkkel             | Verdi      |
|--------------------|------------|
| Elev:1:Fornavn     | Ailin      |
| Elev:1:Etternavn   | Liane      |
| Elev:1:Fødselsdato | 09.10.2018 |
| Elev:2:Fornavn     | Gorm       |
| Elev:2:Etternavn   | Syrstad    |

## Dokumentdatabaser

Data lagres som et antall semistrukturerte dokumenter, gjerne i form av JSON eller XML.

- Hvert dokument har en unik id (nøkkel).
- Velegnet som database bak webløsninger.
- Eksempler: MongoDB, DataBricks.

| Id | Dokument                                                      |
|----|---------------------------------------------------------------|
| 1  | { Fornavn: Ailin, Etternavn: Liane, Fødselsdato: 09.10.2018 } |
| 2  | { Fornavn: Gorm, Etternavn: Syrstad }                         |

### Dokumenter

Et dokument blir representert som et JSON-objekt og organiseres i dokumentsamlinger. En dokumentsamling kan sammenlignes med en tabell i en relasjonsdatabase. Et dokument består av et antall felt, og hvert felt er et nøkkel/verdi-par.

```
{
 VNr: "10830",
 Betegnelse: "Nisseskjegg",
 Pris : 83.00,
 Antall: 42
}
```

### JSON Dokumenter

**JavaScript Object Notation (JSON):** et åpent standard filformat og datautvekslingsformat som bruker menneskelig lesbar tekst til å lagre og overføre dataobjekter som består av attributt-verdi-par.

- Det er raskt og enkelt å analysere.
- Det er språkuavhengig med parsere for mange språk.
- Det brukes til å overføre data mellom en server og et webprogram.
- De grunnleggende datatypene er tall, string, boolsk, array, objekt osv.

## Spørrings mot JSON

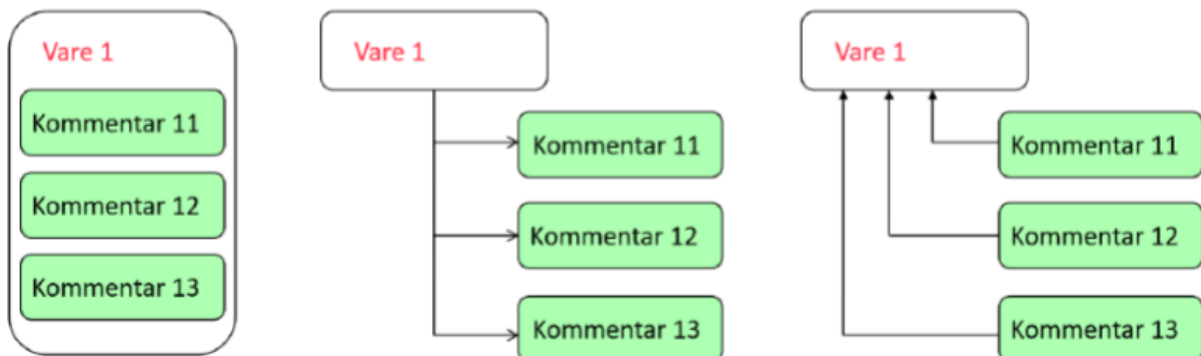
PostgreSQL bruker spesielle operatore for å hente ut data:

| Operator | Hva den gjør                          |
|----------|---------------------------------------|
| →>       | Henter verdi som <b>tekst</b>         |
| →        | Henter verdi som <b>JSON-objekt</b>   |
| #>>      | Henter <b>array-element</b> som tekst |
| #>       | Henter <b>array-element</b> som JSON  |

- Du kan lagre **semistrukturerte data** i en relasjonsdatabase.
- Passer godt når data varierer i struktur.
- Kombinerer fleksibiliteten til NoSQL med styrken til SQL

#### *Hvordan modellere en-til-mange forhold i en dokumentdatabase*

En-til-mange forhold (Eksempelet er: Én vare → mange kommentarer) kan modelleres på tre måter:



| Embedding<br>(Underobjekter)                                                                                                                                                    | Referanser fra<br><b>vare</b> → kommentarer                                                                                                                                      | Referanser fra<br><b>komentarer</b> → vare                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kommentarene ligger inni vare-objektet.                                                                                                                                         | Varen inneholder bare ID-er som peker til kommentarene.                                                                                                                          | Hver kommentar peker tilbake til varen.                                                                                                                      |
| Dette passer når: <ul style="list-style-type: none"> <li>- Når kommentarene <b>alltid brukes sammen med varen.</b></li> <li>- Når kommentarene <b>ikke er store.</b></li> </ul> | Dette passer når: <ul style="list-style-type: none"> <li>- Når kommentarene kan bli <b>mange.</b></li> <li>- Når du noen ganger vil hente <b>komentarer separate.</b></li> </ul> | Dette passer når: <ul style="list-style-type: none"> <li>- Når du ofte henter <b>komentarer alene.</b></li> <li>- Når du vil at kommentarene skal</li> </ul> |

|                                                                                                                                                                                                        |                                                                                                                                                      |                                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- Når du <b>ikke trenger å hente kommentarer alene.</b></li> </ul> <p>Dette gir rask lesing, men kan bli tungt hvis det er <i>veldig mange</i> kommentarer.</p> | <ul style="list-style-type: none"> <li>- Når du vil unngå at vare-objektet blir gigantisk.</li> </ul> <p>Dette ligner mer på relasjonsdatabaser.</p> | <p>være <b>selvstendige objekter.</b></p> <ul style="list-style-type: none"> <li>- Når du ikke trenger å hente alle kommentarer samtidig.</li> </ul> <p>Dette er fleksibelt, men krever mer arbeid for å hente “alle kommentarer til en vare”.</p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Oppsummering

| Modell                          | Fordel                      | Ulempe                           |
|---------------------------------|-----------------------------|----------------------------------|
| <b>Embedding</b>                | Raskt, enkelt               | Blir stort ved mange kommentarer |
| <b>Referanser fra vare</b>      | Mer kontroll, skalerbart    | Krever ekstra oppslag            |
| <b>Referanser fra kommentar</b> | Kommentarer er selvstendige | Vanskeligere å hente alt samlet  |

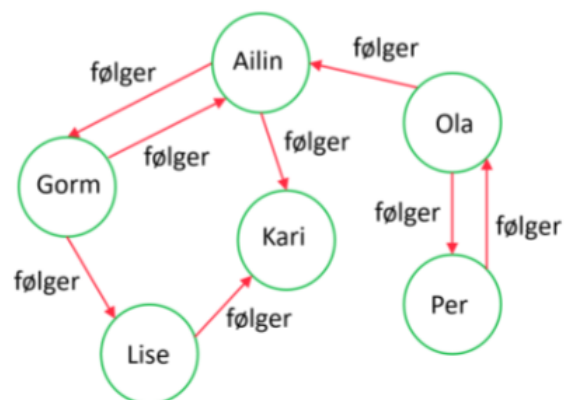
## Grafdatabaser

En grafdatabase lagrer data som **noder** (ting) og **kanter** (forbindelser mellom ting). Tenk på det som et nettverk.

### Når brukes grafdatabaser?

Når data naturlig handler om **relasjoner**, f.eks.:

- hvem som følger hvem
- venner i sosiale medier
- anbefalingssystemer
- nettverk av produkter, personer eller steder



## ER med sammensatte attributter

Sammensatte attributter (objekter, arrays) kan presenteres på denne måten i et ER-diagram (utvidet med komplekse datatyper).

| Vare                        |
|-----------------------------|
| <u>_id</u> : ObjectId (PK)  |
| Betegnelse : String         |
| Pris : Number               |
| Antall : Number             |
| Hylle : String              |
| KatNr : Number              |
| Tagger: Array[String]       |
| Kommentarer : Array[Object] |
| KNr : Integer               |
| Melding : String            |
| Dato : Date                 |